Member-only story

From Subagents to Agent Teams: Claude Code’s Multi-Agent Leap (And What I Actually Change)

I’ve built multi-agent workflows for months now. Agent Teams replaced my custom orchestration in one session — but the limitations are real

9 min read · Feb 7, 2026

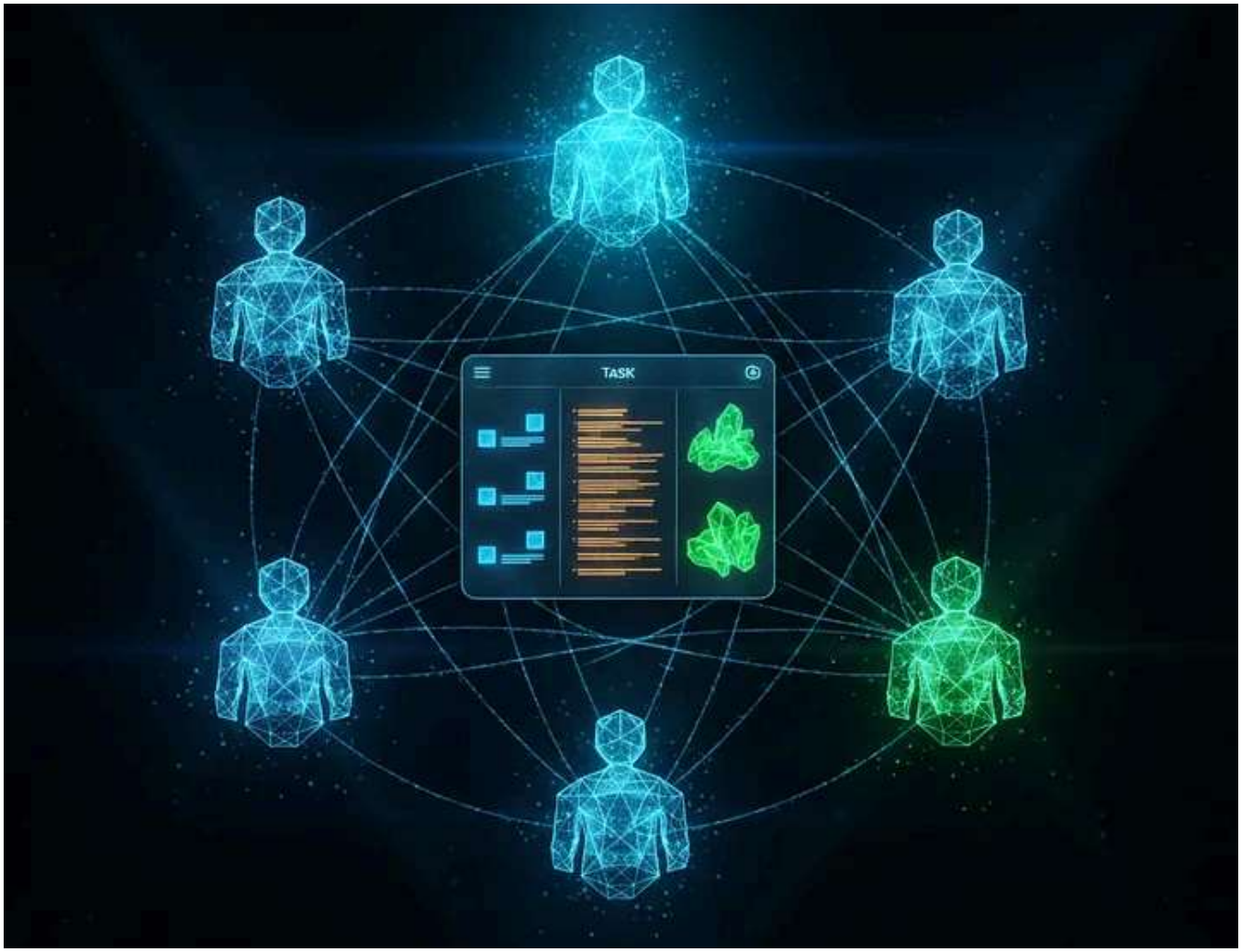
Reza Rezvani

Follow

Listen Share More

The first time I told Claude Code to spawn a team, I expected something like my existing subagent workflows — just faster. Five agents would do what one agent did before, right? More parallel execution, same reporting structure.

That’s not what happened.



Claude Code Agents Team Feature Experience | Image Generated with gemini 3 Pro ©

Note: AI helped do some research for this article. The testing, real observations, and limitations documented here are entirely mine.

I watched the research agent message the explore agent directly. Not through me. Not through a coordinator.

One agent told another:

> *"I found three approaches for this skill pattern — the third one has a dependency issue you should check before you build."*

The explore agent adjusted its approach without me saying a word.

I've been building multi-agent workflows with Claude Code for months. I published a 30 Specialized SubAgents series that became one of my most-read technical pieces. But this was different. Subagents report back to you. Agent Teams talk to *each other*.

The Evolution Nobody Expected This Fast

Let me back up. If you've been following Claude Code's multi-agent story, here's where we were:

Subagents (what most of us have been using) are focused workers. You define them in `~/.claude/agents/` with a Markdown file and YAML frontmatter. They get their own context window, do their job, and report back. The parent agent orchestrates everything. Think of it as a manager delegating tasks to contractors — each contractor works independently, delivers results, goes home.

The limitation? Contractors don't talk to each other. If your security reviewer finds something that affects your backend architect's work, that information routes through you. You become the bottleneck in your own multi-agent system.

Agent Teams *(launched February 5, 2026)* flip this model. Instead of isolated workers reporting to a manager, you get a team with a shared task board and peer-to-peer communication. The Team Lead coordinates, but teammates can message each other, flag dependencies, and adjust their work based on what others discover.

Three components make this work:

1. **Team Lead** — Your main Claude Code session. Plans, delegates, monitors. Doesn't need to do all the work itself.
2. **Teammates** — Independent Claude Code instances, each with their own context window, CLAUDE.md loaded, MCP servers connected, skills available.
3. **Shared Task List** — Tasks with three states: pending → in progress → completed. Supports dependencies — Task B automatically unblocks when Task A completes.

The mental model shift:

> from "contractors on errands" to "a team that collaborates."

Orchestrate teams of Claude Code sessions - Claude Code Docs

Coordinate multiple Claude Code instances working together as a team, with shared tasks, inter-agent messaging, and...

code.claude.com

Setup: Enable Agent Teams in 5 Minutes

This is experimental. Anthropic hasn't hidden that fact — it requires an explicit opt-in flag. Here's how to enable it.

Option 1: Settings file (persistent)

Add to your Claude Code `settings.json`:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Option 2: Environment variable (session-only)

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Prerequisites

- Claude Code (latest version)
- I'm running **Opus 4.6** (released February 5, 2026) — the model matters for coordination quality
- Sufficient API budget — more on the token reality later

Your First Team

Once enabled, creating a team is natural language. No configuration files, no YAML:

```
"Create a team to build a new Claude Code skill for automated
code review. I need one agent to research existing patterns,
```

```
one to explore the best implementation approach, and one to
write the actual skill files."
```

Claude Code spawns the Team Lead (your session), creates the task board, and launches teammates. You'll see task assignments appear in real-time.

Backend Options

Three execution backends control *how* teammates run:

Backend How It Works Best For **In-process** *(default)* Teammates run within the same process Quick tasks, lower resource overhead **tmux split panes** Each teammate gets a visible terminal pane Debugging, watching agents work **iTerm2** macOS only, native tab integration macOS developers who want visual monitoring

For your first session, stick with in-process. Switch to tmux when you want to watch what's actually happening — it's genuinely fascinating to see agents coordinate in real-time across terminal panes.

What I Actually Tested

I'm currently building new Claude Code skills for my open-source project (claude-code-skill-factory). This felt like the right scenario — multiple skills need research, pattern exploration, and implementation, with natural parallelism between them.

My Team Configuration

I spawned a team with specialized roles:

- **Research Agent** — Survey existing skill patterns, analyze documentation, identify best practices
- **Explore Agent** — Test different implementation approaches, validate feasibility
- **Builder Agent** — Write the actual SKILL.md files, Python scripts, and validation logic

What Surprised Me (Good)

The research and explore agents produced impressively good results. Not *"adequate for a first pass"* — genuinely good. The research agent didn't just summarize

documentation; it cross-referenced patterns across multiple skill implementations and identified inconsistencies I hadn't noticed.

The explore agent took those findings and tested approaches I wouldn't have tried myself. When one approach hit a dependency issue, it pivoted to an alternative and messaged the builder agent with context about *why* it chose the fallback — including the failure path. That's the kind of institutional knowledge that normally lives in someone's head and never gets documented.

The peer-to-peer messaging is what makes this fundamentally different from subagents. When the research agent discovered that a particular skill pattern had been deprecated, the explore agent knew about it within minutes — without me relaying the information. In my subagent workflows, that coordination step was always manual.

What Surprised Me (Not Good)

Session isolation is the biggest friction point. Each teammate runs in its own environment. Agent A can't access files that Agent B created. If the explore agent generates a prototype in its workspace, the builder agent can't see it directly. You end up relaying artifacts between agents, which partially defeats the purpose of peer-to-peer collaboration.

This isn't a bug — it's an architectural decision for safety and context management. But it means your team works more like remote developers without a shared repository than co-located engineers with a shared codebase.

Token usage is noticeably higher. I don't have exact multipliers yet — this experiment is still running — but five teammates means five separate context windows, each loading CLAUDE.md, each processing independently. Anthropic's documentation says token usage scales roughly linearly. That tracks with what I'm seeing. For research and exploration tasks, the value justifies the cost. For routine implementation? Probably not.

The Decision Framework: When to Use What

After running agent teams alongside my existing subagent workflows, here's the CTO-level decision matrix I'm using:

Use Agent Teams When:

- **Parallel exploration** — Multiple approaches to the same problem, tested simultaneously
- **Cross-domain coordination** — Security review + performance review + test coverage, where findings in one domain affect another
- **Research-heavy tasks** — When you need multiple perspectives synthesized before building
- **Competing hypotheses** — You don't know the right approach; let agents argue it out

Use Subagents When:

- **Focused, isolated tasks** — Code review on a single file, documentation generation
- **Token-conscious work** — Budget matters; subagents are lighter weight
- **Well-defined scope** — You know exactly what you need; no exploration required
- **Sequential dependencies** — When Step 2 truly can't start until Step 1 finishes

Use Single Session When:

- **Same-file edits** — Multiple agents editing the same file creates merge conflicts
- **Simple tasks** — If it takes one agent 10 minutes, don't spawn five
- **Debugging** — You need iterative back-and-forth, not parallel work
- **Context-heavy work** — When understanding the full picture matters more than speed

The "Agent Swarm Trap"

Not everything needs a team. I see developers already reaching for multi-agent setups for tasks that a single focused session handles better. The overhead of coordination — task management, message routing, context duplication — has a cost.

Rule of thumb: If you can describe the task in one sentence, use one agent. If the task has three or more distinct workstreams that benefit from different perspectives, consider a team.

What Doesn't Work (Yet)

This is experimental software. Treating it as production-ready will frustrate you. Here's what I've hit:

Open in app ↗

Medium

projects.

Task status lag. Teammates sometimes forget to mark tasks as completed. The task board shows *"in progress"* for work that finished minutes ago. If you have dependent tasks, this creates unnecessary delays. Workaround: the Team Lead can prompt teammates to update their status.

One team per session. You cannot run nested teams or multiple teams from the same session. For complex projects that need hierarchical coordination *(team of teams)*, you'll need to restructure your approach into sequential team sessions.

Shutdown is slow. When you end a session, the system waits for each teammate to finish its current request before shutting down. With five teammates, this can take a minute or more.

No shared filesystem. As I mentioned — each agent's workspace is isolated. This is the limitation that generates the most friction in practice. I expect this to evolve, but for now, plan your task decomposition around artifacts that can be communicated via messages rather than shared files.

I'm still figuring out the optimal team size. Three teammates felt productive. Five felt like the coordination overhead started to outweigh the parallelism benefit. Anthropic stress-tested with 16 agents *(their C compiler project — 100,000 lines of Rust, $20,000 in API costs)*, but that's not a typical workflow. For daily development work, I'd start with 3–4 teammates and scale up only if the task genuinely benefits from more parallel workstreams.

What I'm Betting On

Here's my honest strategic read: ***multi-agent orchestration is becoming the default mode for AI-assisted development.*** Not next year. Now.

The progression is clear:

```
Single agent (2024) → Subagents (mid-2025) → Agent Teams (Feb 2026)
```

Every Claude Code user I know has built custom multi-agent workflows — shell scripts spawning multiple instances, tmux sessions with manual coordination, custom orchestration layers. We all invented our own versions of what Anthropic just shipped natively.

That's the signal. When the entire community builds the same workaround, the platform follows. Agent Teams is Anthropic saying: *"We see what you're building. Here's the infrastructure."*

For existing Claude Code users with established subagent workflows: it's time to evolve. Not abandon — evolve. Your subagent definitions still work. Your CLAUDE.md configurations still load. But the coordination layer is now built-in, and fighting that by maintaining custom orchestration scripts will become technical debt.

Anthropic's stress test — 16 agents building a Rust C compiler that compiles the Linux kernel — isn't just a demo. It's a statement of where the ceiling is. Most of us will operate well below that ceiling, but knowing it exists changes how you think about what's possible.

The Honest Assessment

Would I use Agent Teams for everything? No. The session isolation, token cost, and experimental rough edges mean I'm selective about when I reach for it.

Would I go back to subagents-only? Also no. For research-heavy and exploration tasks, watching agents coordinate and challenge each other's findings produces better output than anything I could orchestrate manually.

The truth is somewhere in the middle — and that's fine. The orchestration capabilities are genuinely impressive. The infrastructure isn't fully mature. Both things are true simultaneously.

What I'm testing next: running agent teams on production-ready software, not just open-source skill development. That's where the real stress test happens — tighter constraints, higher stakes, less room for experimental rough edges.

What would you give a team of agents as their first task? I'm curious what scenarios you'd test first.

✨ *Thanks for reading! If you'd like more practical insights on AI development tools and multi-agent systems, hit subscribe to stay updated.*

This article builds on my previous Claude Code multi-agent work. If you're new to subagents, start with 30 Specialized Claude Code SubAgents You Need in 2026 for the foundation.

Claude Code (Claude AI Desktop compatible) Open Source Project on GH:

GitHub - alirezarezvani/claude-skills: A Collection of Skills for Claude Code and Claude AI for...

A Collection of Skills for Claude Code and Claude AI for real-world Usage. Including Claude Code Subagents, Claude Code...

github.com

Or use the following command in Claude Code to install the Plugin:

```
/plugin marketplace add alirezarezvani/claude-skills

# Then install skill bundles:
```

```
/plugin install marketing-skills@claude-code-skills     # 5 marketing skills
/plugin install engineering-skills@claude-code-skills   # 18 engineering skills
/plugin install product-skills@claude-code-skills       # 5 product skills
/plugin install c-level-skills@claude-code-skills       # 2 C-level advisory sk
/plugin install pm-skills@claude-code-skills            # 6 project management
/plugin install ra-qm-skills@claude-code-skills         # 12 regulatory/quality

# Or install individual skills:
/plugin install content-creator@claude-code-skills      # Single skill
/plugin install fullstack-engineer@claude-code-skills   # Single skill
```

About the Author

I'm Alireza Rezvani (Reza), CTO building AI development systems for engineering teams. I write about turning individual expertise into collective infrastructure through practical automation.

Connect: Website | LinkedIn **Read more on Medium:** Alireza Rezvani

Claude Code | Multi Agent Systems | Ai Development | Software Engineering | Programming

Follow

Written by Reza Rezvani

4.2K followers · 76 following

As CTO of a Berlin AI MedTech startup, I tackle daily challenges in healthcare tech. With 2 decades in tech, I drive innovations in human motion analysis.

Responses (4)

Holger Kreissl

What are your thoughts?

Andreas Schlapbach he/him
Feb 7

Bleeding edge research, read of the week.

4 1 reply Reply

Philip Patterson
Feb 7

You have a typo in the repo URL for your skill factory

3 1 reply Reply

Kamrun Nahar she/her
Feb 14

Loved every bit of this article. Keep creating amazing content!

1 1 reply Reply

See all responses

More from Reza Rezvani

Reza Rezvani

I Tested Every Major Claude Opus 4.6 Feature—Here's What Actually Matters

After 24 hours of real testing across my daily workflow, here's my honest calibration on the new Anthropic's Frontier Model

Feb 6 1K 14

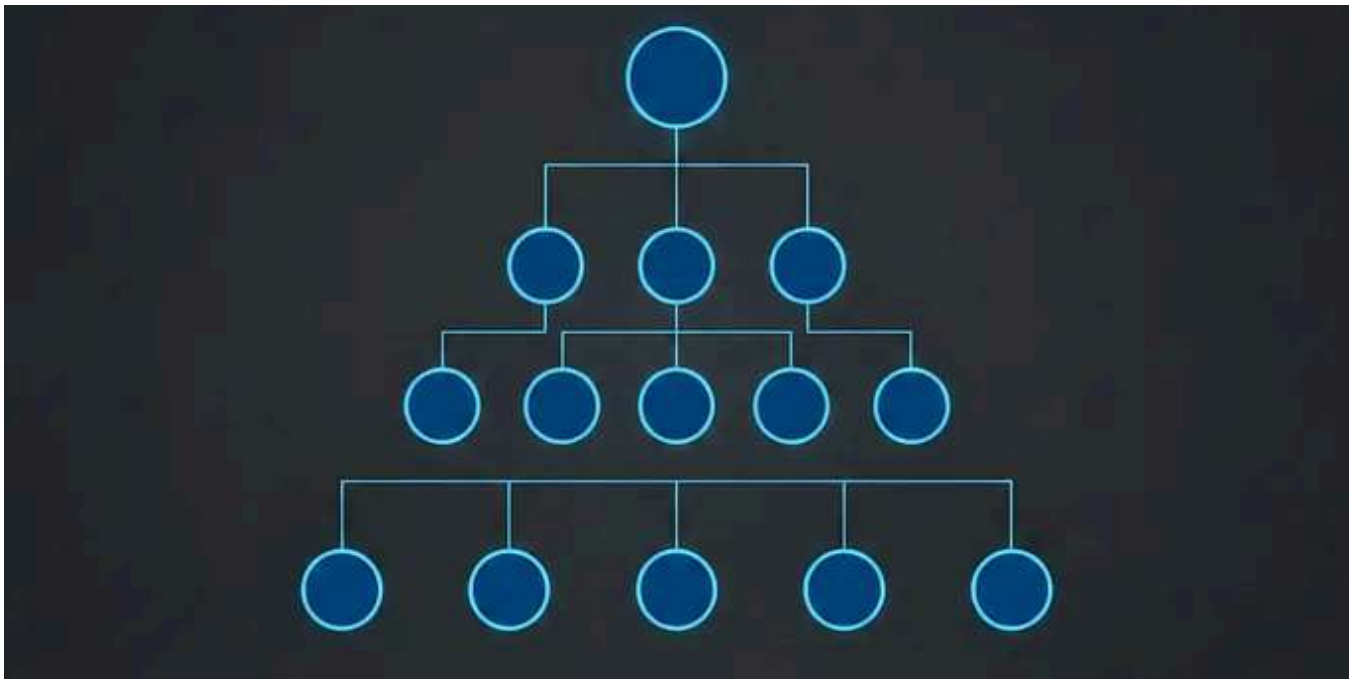
```
Option 1: Continue to Priority 2 (Performance)
- Build /profile command
- Build /benchmark command
- Complete performance category (2 commands, ~24-32 hours)

Option 2: Test Security Commands
- Validate /audit, /vulnerability-scan, /compliance-check in real projects
- Gather feedback
- Refine before continuing

Option 3: Commit Progress
- Save 3/10 security commands to repository
- Document in changelog
- Create v2.7.0 release

Recommendation: Continue to Priority 2: Performance to build /profile and /benchmark commands, completing the next category.

What would you like to do next?
1. Continue to Priority 2 - Build /profile command
2. Test the 3 security commands
3. Commit progress and create release
4. Something else
```

Reza Rezvani

10 Claude Code Commands That Cut My Dev Time 60%: A Practical Guide

Custom slash commands, subagents, and automation workflows that transformed my team's productivity—with copy-paste templates you can use

Nov 20, 2025 1.4K 32



Reza Rezvani

OpenClaw Multi-Agent System: The Blueprint I Built in 12 Hours

From one AI Assistant built with OpenClaw (Moltbot) 13 specialized agents with cron jobs, shared memory, and self-updating kanban — the...

Feb 17 437 9

Reza Rezvani

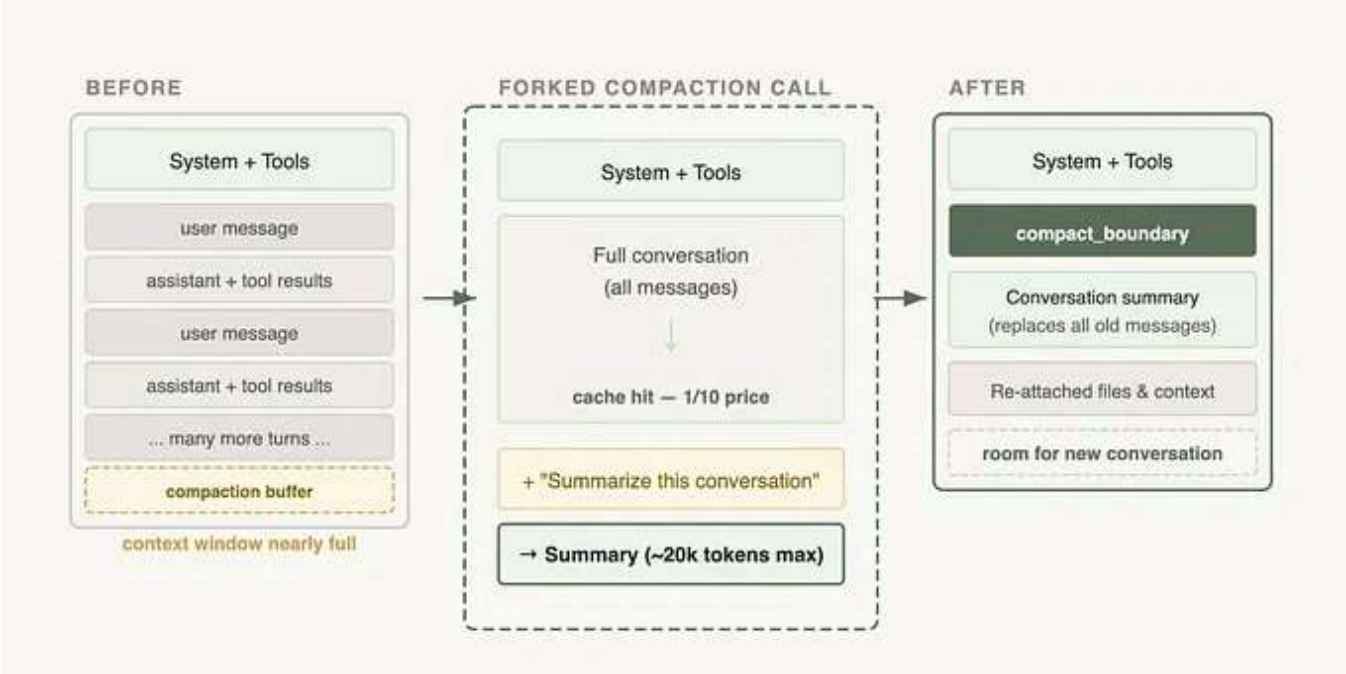
Everyone’s Installing Moltbot (Clawdbot). Here’s Why I’m Not Running It in Production (Yet).

What actually works, what doesn’t, and whether you should set this AI Assistant up

Jan 27 187 9

See all from Reza Rezvani

Recommended from Medium

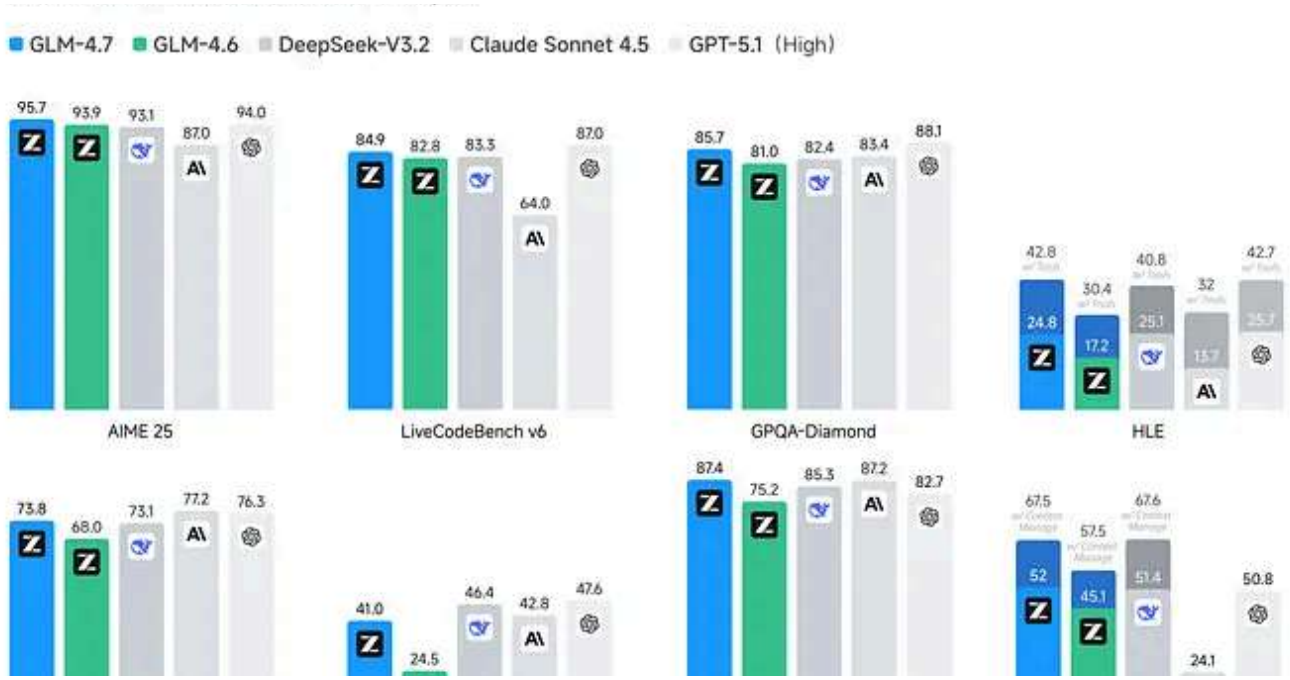


In AI Software Engineer by Joe Njenga

Anthropic Just Fixed the Biggest Hidden Cost in AI Agents (Automatic Prompt Caching)

With just one change, you can cut Claude API costs to 10 cents on the dollar and stop bleeding cash on every single API call.

Feb 20 729 16

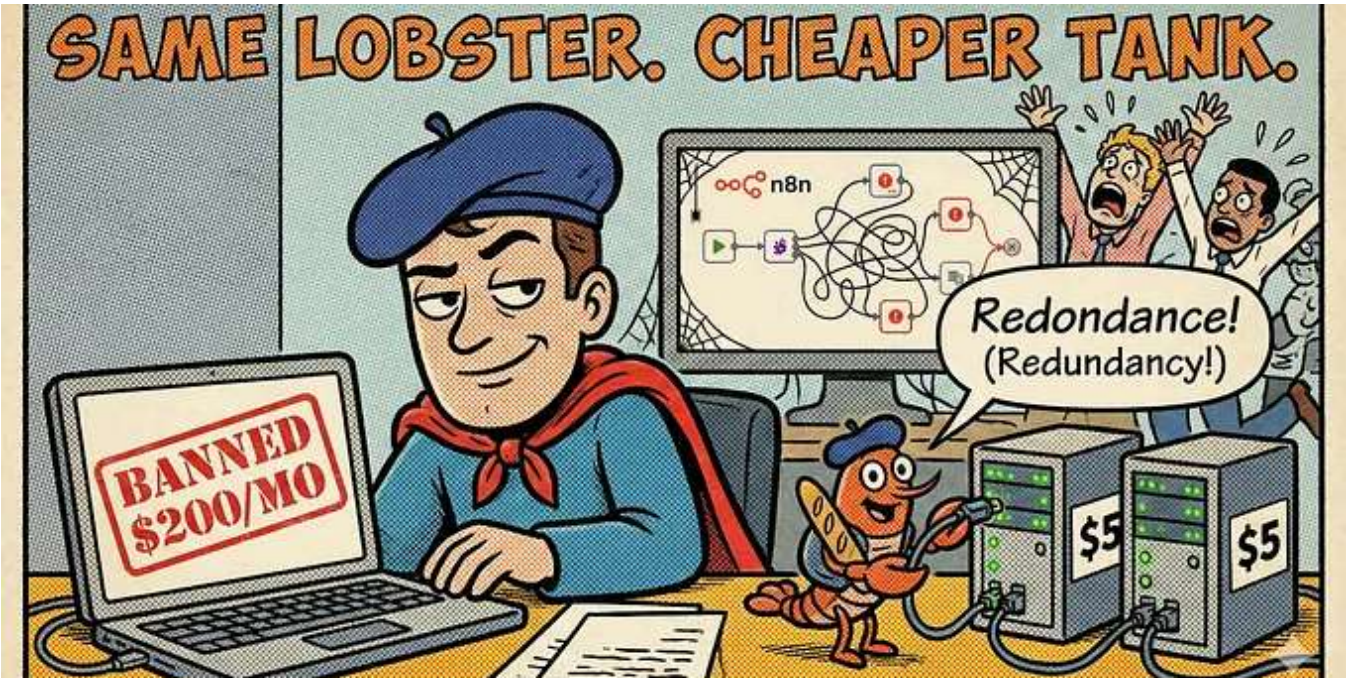


Agent Native

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you might be wondering...

Jan 20 1.5K 42

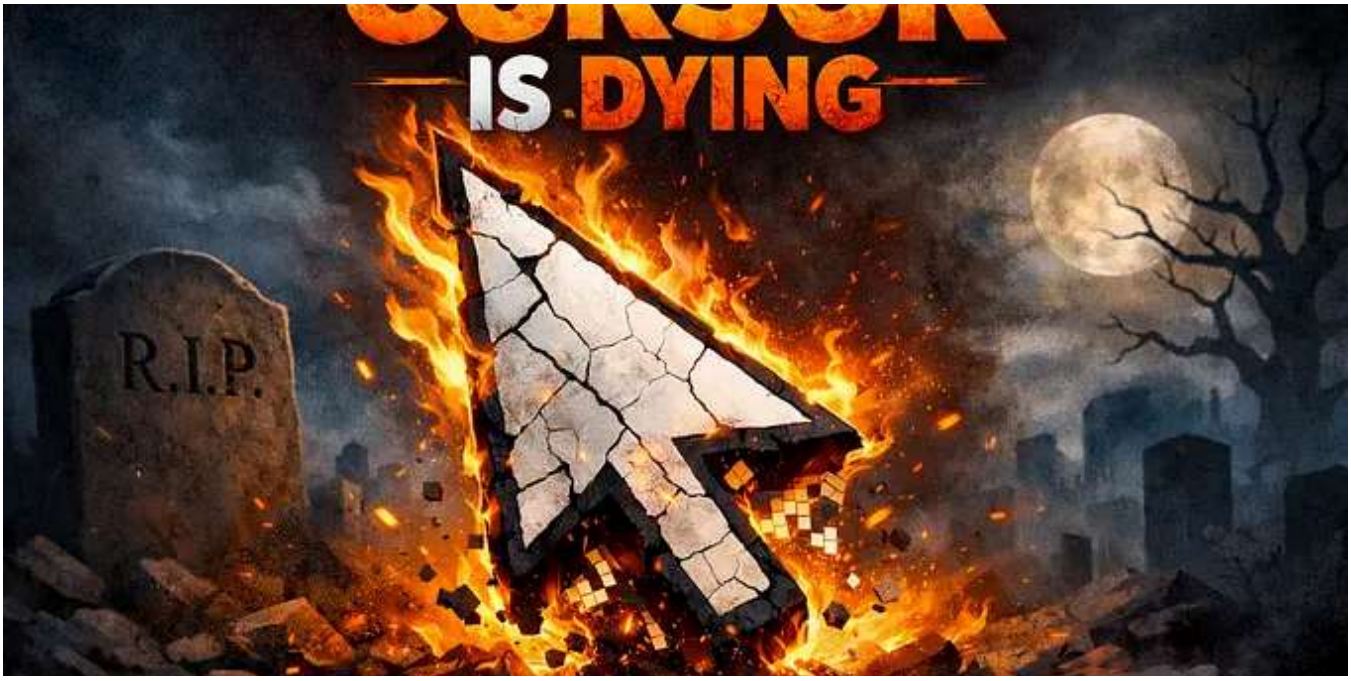


Phil | Rentier Digital Automation

Anthropic Just Killed My $200/Month OpenClaw Setup. So I Rebuilt It for $15.

My OpenClaw instance ran on Claude Max for six weeks. Emails, calendar, Telegram, the whole personal Jarvis fantasy. Then one morning...

Feb 19 765 28

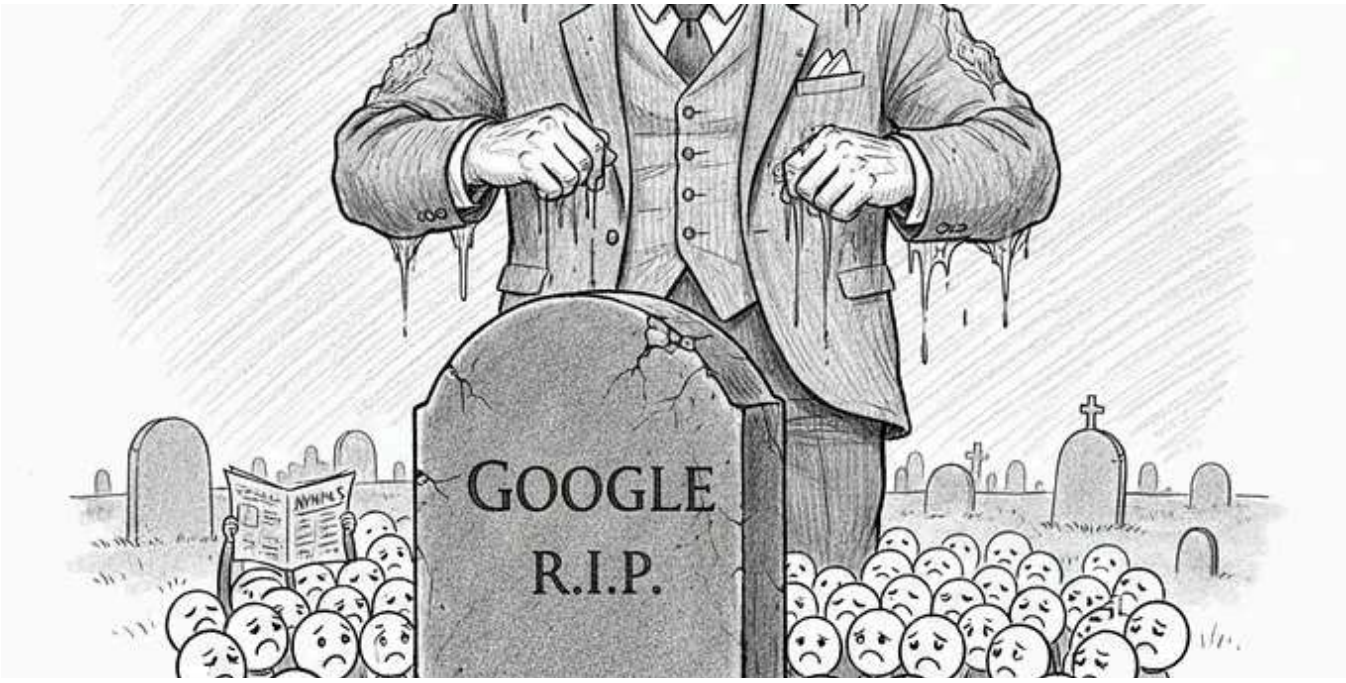


In ITNEXT by Jacob Ferus

Cursor Is Dying

Cursor is a great product. It was one of the first great applications of AI in coding, moving past copy-pasting code from chats. But the AI...

Feb 11 257 10

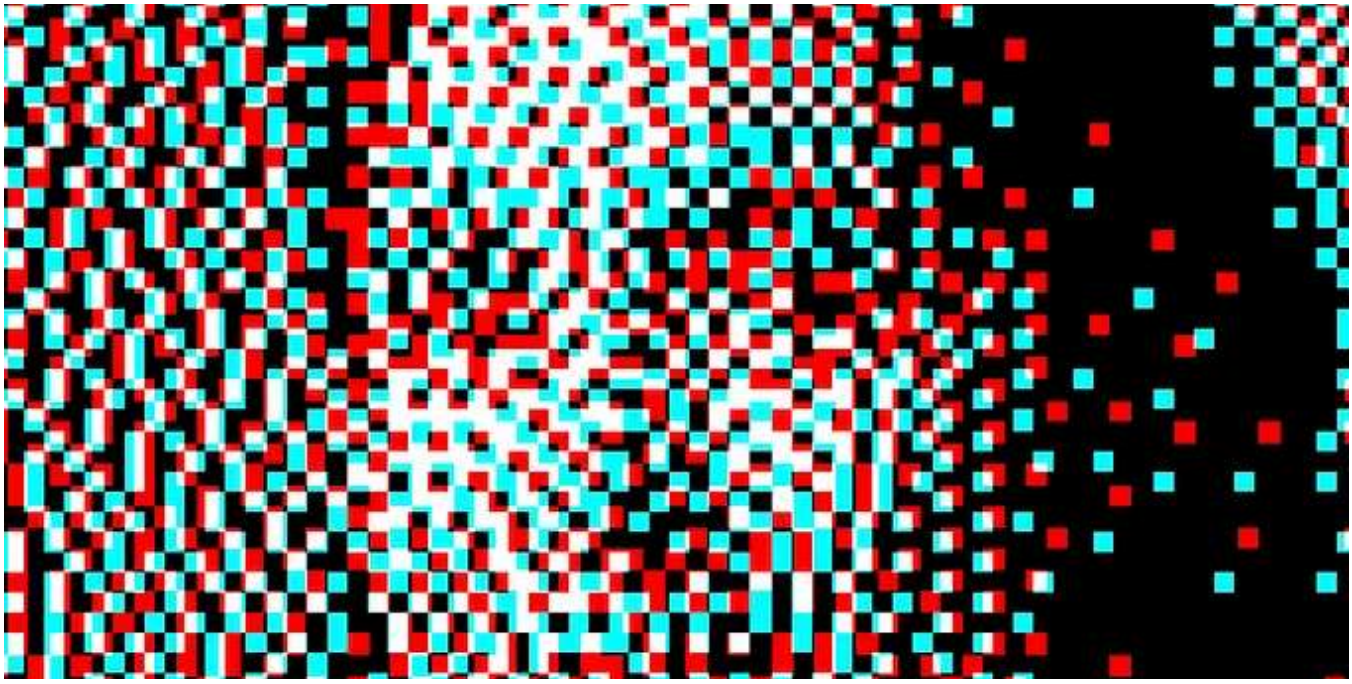


In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine inside someone else's car.

Feb 17 4K 136



Marco Kotrotsos

The Agentic Engineering Playbook.

According to OpenClaw's Creator

Feb 17 210 8

See more recommendations